COMSATS University, Islamabad Pakistan

Aassignment-02

CLO-2

PodCraft – Agentic AI Podcast Episode Planning System
Version 1.0

Submitted By:

Asna Hammad FA23-BDS-051

Awais Ashraf FA23-BDS-053

Supervisor

Dr. Tehseen Riaz Abbasi

Bachelor of Science in Data Science / Software Engineering

(2023-2027)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University, Islamabad Pakistan

PodCraft – Agentic AI Podcast Episode Planning System

A project presented to

COMSATS University, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelors of Science in Data Science / Software Engineering

(2023-2027)

By

Asna Hammad FA23-BDS-051

Awais Ashraf FA23-BDS-053

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Asna Hammad

Awais Ashraf

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (DS) PodCraft was developed by Muhammad **Awais Ashraf (FA23-BDS-053)** and **Asna Hammad (FA23-BDS-051)** under the supervision of **Dr. Tehseen Riaz Abbasi** and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Data Science.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

In the rapidly growing domain of digital content creation, podcasters face significant challenges during pre-production, including time-consuming manual research, unorganized episode structuring, and difficulty in generating engaging questions and hooks. Appointing manual methods for this task involves many problems such as increased preparation time, inconsistency in content quality, lack of structured workflows, no intelligent assistance, and reliance on scattered digital tools. Talking about current planning methods, they use general note-taking applications or basic script editors irrespective of the fact that podcasters need automated research and structured outlines.

To prevent all these problems from prevailing, PodCraft is developed. It is an agentic AI-based podcast planning system, which provides solutions to many of these problems. It is a cross-platform application (web and mobile) which allows podcasters to input their niche or guest information. It monitors different content requirements using intelligent autonomous agents which can research topics based on user input and generate structured episode plans for later reference as well.

PodCraft is a podcast planning system that can be assigned to a content type, as in our case podcast episodes, it works accordingly, analyzes user intent upon receiving niche or guest details, and performs user-defined actions like generating questions, hooks, timestamps, and complete episode outlines for future use.

It is an intelligent system consisting of a React JS web frontend, React Native mobile app, Node JS backend with Express, and MongoDB database. OpenAI API is used for AI agent processing, and Firebase is used for cloud authentication and real-time synchronization. LAN and internet communication are used by users to access the platform and operate the system manually or through automated agents.

Abstract

PodCraft is developed to simplify and automate podcast episode planning using intelligent autonomous agents.
Currently, podcasters rely heavily on manual research, scattered online sources, and unstructured planning methods, which consume significant time and often reduce content quality.
Many creators struggle to organize episode flow, prepare engaging questions, and design strong introductions that retain audience attention. This leads to inconsistent production quality and inefficient preparation workflows.
There is no unified intelligent platform that autonomously plans podcast episodes based on niche and guest information.
PodCraft addresses this gap by using autonomous agents to research topics, generate structured questions, create engaging hooks, and design complete episode outlines. The system aims to automate preparation, improve episode structure, and assist creators in producing professional content efficiently. It also enhances time management by providing timestamped segments and guided content flow.
The project ultimately enables podcasters to focus more on creativity and delivery rather than planning logistics. PodCraft improves productivity and raises the overall quality of podcast production through intelligent automation.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor Dr. Tehseen Riaz Abbasi. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work.

Asna Hammad

Awais Ashraf

Abbreviations

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
FE	Feature
FR	Functional Requirement
JWT	JSON Web Token
LI	Limitation
MVC	Model-View-Controller
SRS	Software Requirements Specification
UI	User Interface
UX	User Experience
WBS	Work Breakdown Structure

Table of Contents

Abstract v

1 Chapter 1: Introduction and Problem Definition 1

1.1 Overview of the Project 1
1.2 Vision Statement 1
1.3 Problem Statement 2
1.4 Problem Solution 2
1.5 Objectives of the Proposed System 3
1.6 Scope 3
1.6.1 Limitations/Constraints 4
1.7 Modules 4
1.7.1 Client Web Application 4
1.7.2 Client Mobile Application 6
1.7.3 Admin Web Application 6
1.8 Related System Analysis/Literature Review 7
1.7.1 Literature Review 7
1.7.2 Related System Analysis 8
1.9 Tools and Technologies 8
1.10 Project Contribution 9
1.11 Relevance to Course Modules 10
Chapter 2: Requirement Analysis 12
2.1 User Classes and Characteristics 12
2.2 Operating Environment 12
2.3 Design and Implementation Constraints 13
2.4 Requirement Identifying Technique 14
2.5 Use Case Diagram (also Description) 21
2.5.1 Use Case Lists (Module-wise) 22
Module 1: User Authentication & Profile Management (Web) 22
2.5.2 Use Cases (Tabular Format) 24
2.6 Functional Requirements 35
2.7 Non-Functional Requirements 40
2.7.1 Reliability 40
2.7.2 Usability 40
2.7.3 Performance 41
2.7.4 Security 41
2.8 External Interface Requirements 41
2.8.1 User Interface Requirements 41
2.8.2 Software Interfaces 42
2.8.3 Hardware Interfaces 42
2.8.4 Communications Interfaces 43

References 43

List of Figures:

Figure 2.5: Use case diagram……21

1 Chapter 1: Introduction and Problem Definition

This chapter provides an overview of the PodCraft project, including its vision, problem context, proposed solution, objectives, scope, modules, literature review, tools, contributions, and relevance to coursework.

1.1 Overview of the Project

PodCraft is an intelligent, agentic AI-based podcast episode planning system designed to automate and streamline the pre-production phase of podcast creation. The system operates within the digital content creation domain, targeting independent podcasters, professional content producers, and small production teams who struggle with time-consuming manual research and unstructured planning workflows. The system bridges the gap between creative ideation and structured execution by employing autonomous AI agents to research topics, analyze guest backgrounds, generate interview questions, create engaging hooks, and produce complete episode outlines with timestamped segments. The proposed solution offers a scalable, cross-platform interface (web and mobile) that reduces preparation time, improves episode consistency, and allows creators to focus more on delivery and audience engagement rather than logistical planning.

1.2 Vision Statement

For podcast creators and content producers who need faster and more efficient ways to plan structured and engaging podcast episodes, the PodCraft is an agentic artificial intelligence-based podcast planning system that autonomously researches topics, generates interview questions, creates engaging hooks, assigns timestamps, and organizes complete episode structures. Unlike manual planning methods and general content tools that require scattered effort and repeated user input, our product provides a fully integrated intelligent agent workflow that automates podcast preparation and improves content quality while saving time and effort.

1.3 Problem Statement

Podcast content creators spend a significant amount of time researching topics, analyzing guest backgrounds, and structuring episode flow before recording. This preparation process is often unorganized and depends heavily on manual internet searches, personal notes, and scattered digital tools. Creators frequently struggle to identify trending and relevant topics that match their podcast niche. Preparing meaningful and engaging interview questions requires deep understanding of the guest's expertise and audience expectations. Many podcasters also face difficulty in organizing episode segments and managing time distribution effectively. As a result, episodes may become inconsistent, poorly structured, or less engaging for listeners. Creating attractive introductions and hooks is another challenge that directly affects audience retention. Current podcast planning tools mainly offer simple text editing or script writing features but lack intelligent research and automated structuring capabilities. There is no integrated system that autonomously converts a podcast idea into a complete episode plan. The absence of intelligent planning support increases workload and reduces creative efficiency. This problem becomes

more significant for independent creators and small production teams with limited resources. Additionally, inefficient planning leads to longer preparation time and delays in publishing schedules. The growing demand for high-quality content makes manual planning methods insufficient. Therefore, an intelligent automated system is required to simplify podcast episode preparation and improve production efficiency.

1.4 Problem Solution

The proposed system, PodCraft, introduces an intelligent agent-based platform that automates podcast episode planning. The system allows users to provide their podcast niche or guest information as input. A User Intent and Guest Analyzer agent interprets this input to determine content direction and audience expectations. A Topic Research Agent collects relevant themes, trending discussions, and important background information related to the niche or guest. The Question and Timestamp Generator agent produces structured interview questions and assigns estimated time durations for each segment. A Hook and Intro Suggestion Agent generates engaging opening statements designed to capture listener attention. The Episode Outline Planner agent organizes all generated content into a logical and smooth episode structure. This integrated workflow reduces manual effort and improves planning efficiency. The system ensures that episodes follow a professional structure with balanced content distribution. Intelligent automation allows creators to focus more on presentation skills and creativity. The platform also improves consistency in episode quality across different productions. PodCraft supports both web and smartphone platforms for flexible accessibility. The system is designed with a user-friendly interface to support both beginner and professional podcasters. Overall, the proposed solution enhances productivity, reduces preparation time, and improves content organization through artificial intelligence automation.

1.5 Objectives of the Proposed System

BO-1: Reduce podcast episode planning time by at least 50%.

BO-2: Improve episode content structure and organization quality by 40%.

BO-3: Increase audience engagement through improved hooks and structured discussions.

BO-4: Automate topic research and question generation to reduce manual workload.

BO-5: Provide an integrated platform for podcast planning across web and smartphone devices.

1.6 Scope

The scope of PodCraft includes the design and development of an intelligent podcast episode planning system that operates on both web and smartphone platforms. The system focuses on assisting podcast creators during the pre-production phase by automating research and planning activities. Users will be able to provide podcast niche details or guest information as primary inputs. The system will analyze this input to determine audience interests and content direction. PodCraft will include autonomous agents responsible for topic discovery and background research. The system will generate structured interview questions tailored to the podcast theme and guest expertise. It will also allocate estimated timestamps for each segment to improve

episode time management. An intelligent module will create engaging hooks and introduction suggestions to capture listener attention. The system will organize all generated content into a coherent and professional episode outline. Users will be able to review, edit, and export the generated episode plan. The platform will provide a simple and user-friendly interface suitable for both beginner and professional podcasters. PodCraft will support secure user authentication and cloud-based data storage for saving episode plans. The system will not include audio recording, editing, or publishing features, as those functions belong to post-production tools. It will not replace podcast hosting platforms but will complement them by improving preparation efficiency. The project is limited to automated planning and structuring rather than full podcast production. The system boundaries focus strictly on intelligent content preparation and workflow automation. Overall, PodCraft aims to enhance productivity and planning quality within the defined pre-production scope.

1.6.1 Limitations/Constraints

LI-1: The system depends on internet connectivity for topic research and cloud data storage.

LI-2: AI-generated content may require manual review to ensure accuracy and relevance.

LI-3: The system does not support podcast audio recording or editing features.

LI-4: Podcast publishing and distribution are outside the system scope.

LI-5: Topic research accuracy depends on available online information sources.

LI-6: Real-time trend analysis may be limited by data access restrictions.

LI-7: The system performance may vary depending on device specifications and network speed.

1.7 Modules

The proposed project consists of multiple modules categorized according to system platforms.

1.7.1 Client Web Application

Module 1.1: User Authentication & Profile Management

FE-1: User registration and secure login

FE-2: Profile creation and podcast niche selection

FE-3: Manage guest profiles and episode history

FE-4: Secure cloud storage of user data

Module 1.2: User Intent & Guest Analyzer

FE-1: Accept podcast niche input

FE-2: Accept guest professional background details

FE-3: Analyze audience interests and content direction

FE-4: Generate content planning recommendations

Module 1.3: Topic Research Agent

FE-1: Discover trending podcast topics

FE-2: Collect background information related to niche

FE-3: Identify audience-relevant discussion themes

FE-4: Provide summarized research insights

Module 1.4: Question & Timestamp Generator

FE-1: Generate structured interview questions

FE-2: Categorize questions by segment type

FE-3: Allocate estimated timestamps to each segment

FE-4: Balance discussion flow across the episode

Module 1.5: Hook & Intro Suggestion Agent

FE-1: Generate attention-grabbing opening hooks

FE-2: Provide multiple introduction tone options

FE-3: Customize hooks based on target audience

FE-4: Suggest episode titles

Module 1.6: Episode Outline Planner

FE-1: Organize content into logical episode flow

FE-2: Divide episode into structured segments

FE-3: Provide editable episode templates

FE-4: Export outline in PDF or document format

1.7.2 Client Mobile Application

Module 2.1: Mobile User Dashboard

FE-1: View saved episode plans

FE-2: Quick input for niche or guest details

FE-3: Real-time synchronization with web platform

Module 2.2: Quick Plan Generator

FE-1: Generate episode outline instantly

FE-2: View summarized topic research

FE-3: Access pre-generated questions and hooks

Module 2.3: Plan Editing & Export

FE-1: Edit episode structure

FE-2: Modify timestamps and segments

FE-3: Export plan to mobile storage or share

1.7.3 Admin Web Application

Module 3.1: User Management

FE-1: Manage registered users

FE-2: Monitor user activity

FE-3: Suspend or remove accounts

Module 3.2: Content Monitoring

FE-1: Monitor AI-generated content quality

FE-2: Manage flagged or inappropriate content

FE-3: Maintain system content standards

Module 3.3: System Configuration

FE-1: Manage AI agent configurations

FE-2: Update topic research sources

FE-3: Monitor system performance

1.8 Related System Analysis/Literature Review

1.8.1 Literature Review

Previous research in intelligent content generation and AI-assisted media production (e.g., Roy & Das, 2021; Zhang & Lee, 2022) emphasizes the role of natural language processing and autonomous agents in reducing manual effort and enhancing creative workflows. However, many existing systems still rely on general-purpose AI tools and lack specialized planning features for podcast production. Our project addresses this gap by integrating agentic AI architecture with domain-specific podcast planning features to ensure accurate, structured, and engaging episode outlines.

Similar approaches have been explored by Kim & Robinson (2020), who applied intelligent agents for automated content structuring, and Williams (2021), who experimented with AI-based conversational systems for creative industries. While these studies highlight the potential of AI in content creation, they often neglect podcast-specific requirements such as timestamp allocation, guest analysis, and hook generation. Our system extends the literature by combining autonomous research agents with structured episode planning algorithms, offering a holistic podcast pre-production solution.

1.8.2 Related System Analysis

Application Name	Weakness	Proposed Project Solution
Descript	Focuses only on editing and post-production; no automated episode planning support	PodCraft provides automated topic research, structured planning, and complete episode outlines before recording
Buzzsprout	Only supports hosting and distribution; lacks content preparation tools	PodCraft assists in pre-production planning through intelligent agents that generate questions, hooks, and timestamps
Notion	Manual planning tool with no intelligent automation	PodCraft uses autonomous AI agents to automatically convert ideas into structured episode plans
ChatGPT	General-purpose AI requiring repeated prompts; no structured workflow	PodCraft provides a dedicated podcast-planning workflow with specialized agents for each task

Table 1.1 Related System Analysis with proposed project solution

1.9 Tools and Technologies

Category	Tool/Technology	Purpose	Rationale
Tools	Visual Studio Code	Integrated Development Environment	Lightweight, extensive extensions, Git integration
	MongoDB	NoSQL Database	Flexible schema for user data and episode plans
	Firebase	Cloud Authentication & Database	Real-time sync and secure authentication
	Figma	UI/UX Design	Collaborative prototyping and mockup creation
	Postman	API Testing	Backend service validation
	GitHub	Version Control	Team collaboration and code management
	Android Studio	Mobile Development	React Native emulation and debugging
	Trello	Project Management	Task tracking and progress monitoring
Technologies	Python 3.11	Backend & AI Processing	AI agent development and data processing
	JavaScript (ES6)	Frontend & Backend Scripting	Full-stack development consistency
	TypeScript 5.3	Frontend Development	Type-safe code and better maintainability

	React JS 18.2	Web Frontend Framework	Component-based architecture, high performance
	React Native 0.72	Mobile App Framework	Cross-platform (iOS/Android) from single codebase
	Node JS 20.10	Server-side Runtime	Scalable, event-driven architecture
	Express JS 4.18	Backend Web Framework	REST API creation and middleware handling
	OpenAI API	AI Content Generation	State-of-the-art language model for agent tasks
	REST API	Frontend-Backend Communication	Standardized, stateless communication
	Tailwind CSS 3.4	UI Styling	Utility-first, responsive design
	JWT	Secure Authentication	Stateless, secure user sessions
	Cloudinary API	Media Storage	Image and document optimization

Table 1.2 Tools and Technologies for Proposed Project

1.10 Project Contribution

The PodCraft system introduces several technical and conceptual contributions compared to existing manual and digital podcast planning solutions:

- **Agentic AI Architecture:** Uses specialized autonomous agents (User Intent Analyzer, Topic Research Agent, Question Generator, Hook Generator, Outline Planner) that collaborate to produce complete episode plans, unlike single-prompt general AI tools.

- **Domain-Specific Podcast Planning:** Provides dedicated features for podcast pre-production including timestamp allocation, guest background analysis, and segment categorization that generic content tools lack.

- **Automated Topic Research:** Employs intelligent web research capabilities to discover trending and relevant topics based on podcast niche and guest profile.

- **Structured Question Generation:** Produces categorized, context-aware interview questions organized by segment type with estimated time durations.

- **Engaging Hook Creation:** Generates attention-grabbing introductions with multiple tone options customizable for different audience demographics.

- **Cross-Platform Accessibility:** Offers both web and mobile applications with real-time synchronization, enabling flexible workflow integration.

- **Editable Episode Outlines:** Provides exportable, modifiable templates that balance automation with creative control.

Contribution Impact:

By integrating agentic AI, natural language processing, and cross-platform development, the system transforms a simple note-taking approach into a smart, adaptive podcast planning ecosystem. These contributions significantly improve efficiency, consistency, engagement quality, and production speed compared to conventional manual planning methods.

1.11 Relevance to Course Modules

- **Software Engineering:** Applied SDLC phases, requirement gathering, design modeling, and testing methodologies.
- **Data Science & AI Integration:** Implemented OpenAI API for content generation, agent-based reasoning, and natural language understanding.
- **Database Systems:** Structured and unstructured data storage using MongoDB for user profiles, episode plans, and guest information.
- **Web Development:** Developed web application using React JS, Node JS, Express, and Tailwind CSS.
- **Mobile App Development:** Implemented cross-platform mobile application using React Native.
- **Cloud Computing:** Integrated Firebase for authentication and real-time database synchronization.
- **Cybersecurity:** Implemented JWT-based secure authentication and data encryption for user privacy.

UI/UX Design: Created responsive, user-friendly interfaces using Figma prototyping and Tailwind CSS.

-

Chapter 2: Requirement Analysis

This chapter presents a comprehensive requirement analysis for PodCraft, an agentic AI-powered podcast planning tool. The analysis covers user classes, operating environments, constraints, use case modeling, and both functional and non-functional requirements. The chapter is structured to provide software developers and stakeholders with a complete understanding of what the system must do and how it must behave.

2.1 User Classes and Characteristics

PodCraft serves a diverse set of users who interact with the system in different capacities. The following table identifies each user class and describes its pertinent characteristics.

User Class	Description & Characteristics
Independent Podcaster	A solo content creator who produces a podcast independently. Typically a non-technical user aged 18–45 with basic digital literacy. Uses PodCraft primarily for episode research, question generation, and outline creation. Expects a simple, guided interface and minimal setup. Estimated to represent ~65% of users.
Podcast Production Team	A team of 2–10 professionals including a host, producer, and researcher. Requires collaboration features, multi-episode planning, and advanced export options. More technically proficient than independent podcasters. Accounts for approximately 20% of the user base.
Guest / Interview Subject	A person invited to be interviewed on the podcast. Interacts with the system only when the host shares a generated briefing document or question list. Minimal direct system interaction; primarily a recipient of AI-generated outputs.
System Administrator	A technical user responsible for platform management, user account administration, API key management, and system monitoring. Has full backend access. Requires advanced technical knowledge and is expected to use admin panel exclusively.
Registered Free User	A user who has registered but operates under the free tier with limited monthly episode plans (up to 3 per month). Access to basic topic research and outline generation only; advanced features such as hook suggestions and timestamp generation are restricted.

2.2 Operating Environment

PodCraft is a cross-platform agentic AI application accessible through both a responsive web interface and a native mobile application. The following constraints define the operating environments supported:

- OE-1: The web application shall operate correctly on Google Chrome (v110+), Mozilla Firefox (v108+), Microsoft Edge (v110+), and Apple Safari (v16+) on Windows 10/11, macOS 12+, and Ubuntu 20.04+ operating systems.

- OE-2: The mobile application shall be compatible with Android 10.0 (API level 29) and above, and iOS 14.0 and above, running on smartphones with a minimum screen resolution of 360 x 640 pixels.
- OE-3: The backend server infrastructure shall be hosted on cloud platforms (AWS / GCP / Azure), operating on Linux-based server environments with Node.js 18+ or Python 3.10+ runtime.
- OE-4: The system shall rely on external AI APIs (e.g., OpenAI GPT-4, Perplexity API) for natural language generation and real-time topic research. A stable internet connection with a minimum bandwidth of 5 Mbps is required for optimal performance.
- OE-5: All databases shall be hosted on cloud-managed PostgreSQL instances. The system shall store user data in geographically distributed nodes complying with GDPR and local data protection regulations.
- OE-6: The system shall support real-time agent orchestration via a task queue (e.g., Redis or Celery), capable of handling a minimum of 500 concurrent agent workflows.

2.3 Design and Implementation Constraints

The following constraints will restrict the options available to developers during system design and implementation:

- CON-1: The system shall use OpenAI's GPT-4 API (or an equivalent large language model) as the primary AI engine for all natural language generation tasks including question generation, hook suggestions, and episode outlines.
- CON-2: The frontend web application shall be developed using React.js (v18+) with Tailwind CSS for consistent UI styling across all screen sizes.
- CON-3: The mobile application shall be developed using React Native or Flutter to support both Android and iOS from a single codebase.
- CON-4: All API communications shall use RESTful JSON-based interfaces over HTTPS. WebSocket connections shall be used exclusively for real-time agent progress updates.
- CON-5: User authentication shall be implemented using OAuth 2.0 with JWT token-based session management. Third-party sign-in via Google and Apple ID shall be supported.
- CON-6: All user-generated and AI-generated content shall be stored in a PostgreSQL relational database. AI output caching shall use Redis to minimize redundant API calls.
- CON-7: The agentic task orchestration layer shall be implemented using LangChain or AutoGen framework to manage multi-step agent workflows.
- CON-8: Online payments for subscription plans shall be processed exclusively through Stripe. No direct credit card data shall be stored on PodCraft servers.

2.4 Requirement Identifying Technique

For the PodCraft system, Mockup-Based Requirement Analysis is used to identify and document functional requirements.

In this approach, user interface mockups are designed to visually represent how users will interact with the system. These mockups help in understanding system behavior and extracting functional requirements based on user actions.

The following steps were followed:

- Key features of PodCraft were identified and divided into multiple screens such as:
  - **M1: Login/Signup Screen**
  - **M2: Dashboard**
  - **M3: Topic Generation Screen**
  - **M4: Question & Timestamp Generator**
  - **M5: Episode Outline Planner**
- Each screen was assigned to a unique Mockup ID for easy reference.
- Mockups were conceptually designed using UI/UX tools (e.g., Figma) to represent user interaction.
- Interactive elements such as buttons (Generate, Save, Export), input fields (niche, guest details), and navigation menus were identified.
- Functional requirements were derived from each interaction. For example:
  - Clicking “Generate Topics” triggers topic generation.
  - Entering guest details triggers profile analysis.
- All identified functional requirements were organized in a structured tabular format as defined in Section 2.6.

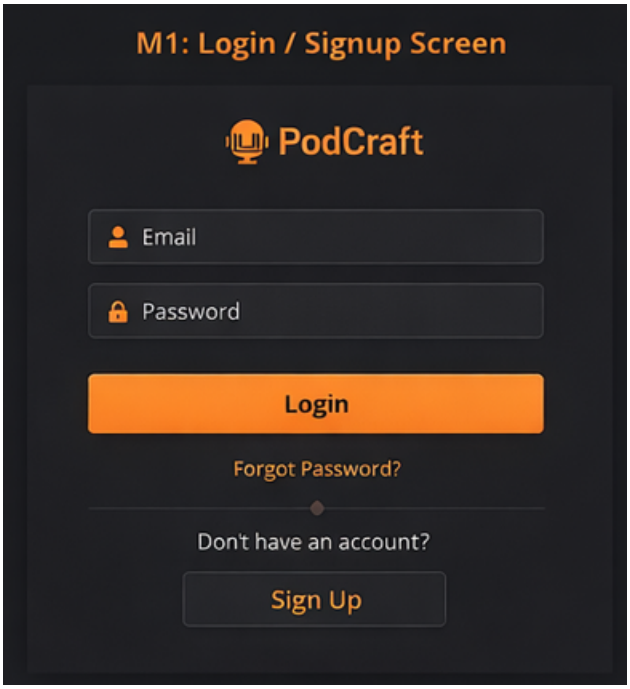
Additionally, backend processes such as AI-based content generation and data retrieval were identified using the Event–Response technique, ensuring that system behavior is captured even when no direct user interaction is involved.

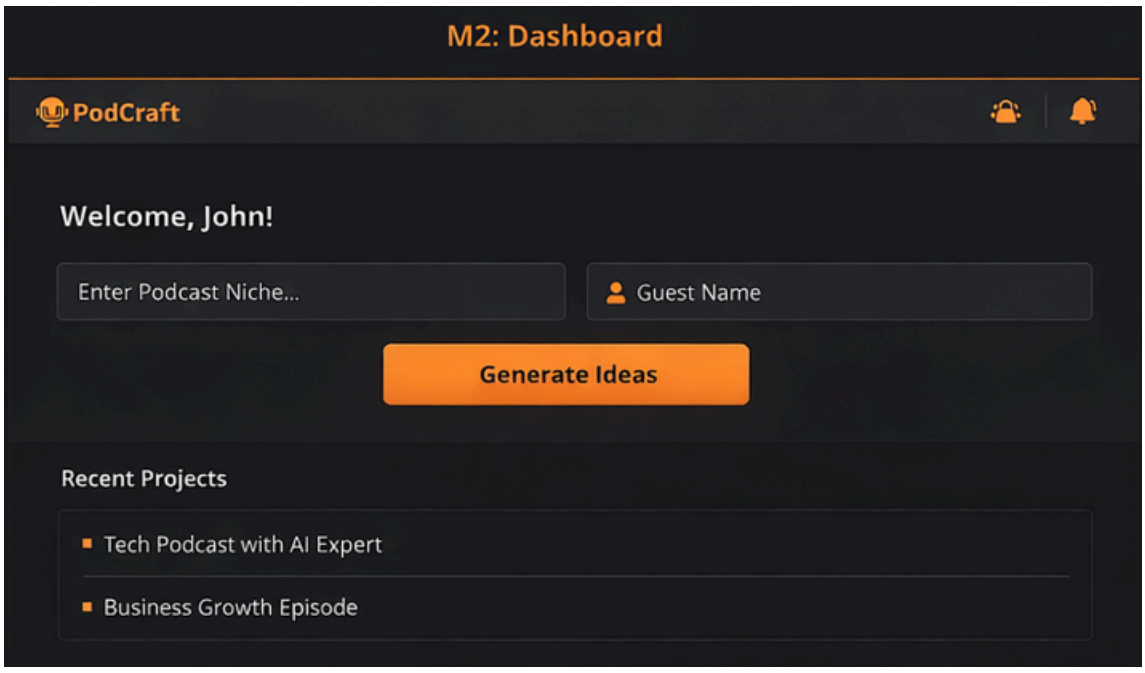
Event Response Table

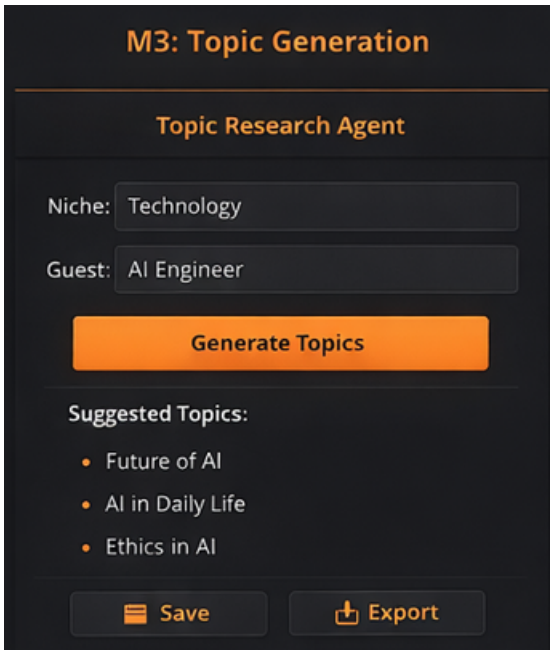
Event Name	Trigger	Source	System State (Before)	Response	System State (After)	Exception Handling
User Login Success	Valid credentials entered	User	Login request pending	1. Grant access 2. Create session 3. Load dashboard	User logged in	None
User Login Failure	Invalid credentials	User	Login request pending	1. Deny access 2. Show error	Login failed	Lock account after 5 attempts

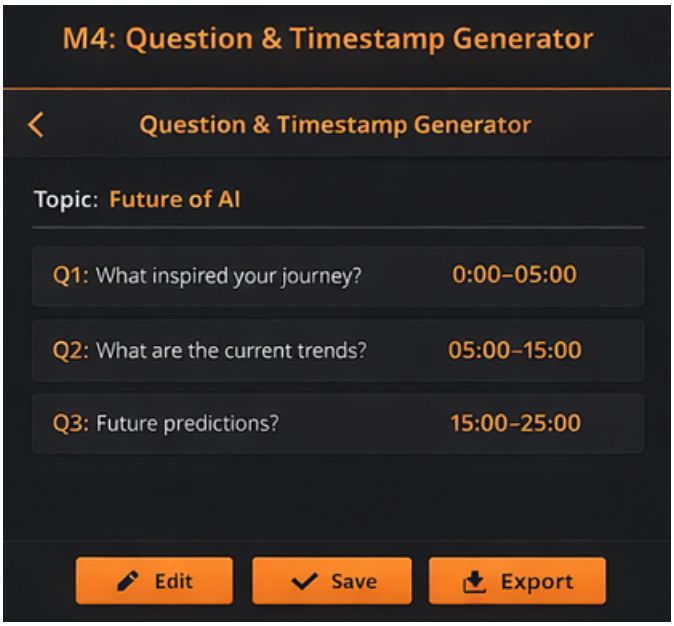
				3. Increment attempt counter		
Niche/Guest Input Submitted	User enters input	User	Dashboard idle	1. Store input 2. Prepare AI request	Input stored	Validate empty input
Topic Generation Requested	Click "Generate Topics"	User	Input available	1. Send request to AI 2. Start processing	Processing state	Retry if request fails
AI Topic Response Received	AI returns topics	AI System	Processing state	1. Display topics 2. Enable save/export	Topics displayed	Show error if response invalid
Question Generation Requested	Click "Generate Questions"	User	Topics available	1. Generate structured questions 2. Assign timestamps	Questions generated	Retry generation
Episode Outline Generated	Click "Generate Outline"	System	Questions ready	1. Create episode structure 2. Add intro/hook	Outline ready	Show fallback structure
System Error Occurred	API/AI failure	System	Any state	1. Display error message 2. Log error	Error state	Retry option + notify admin

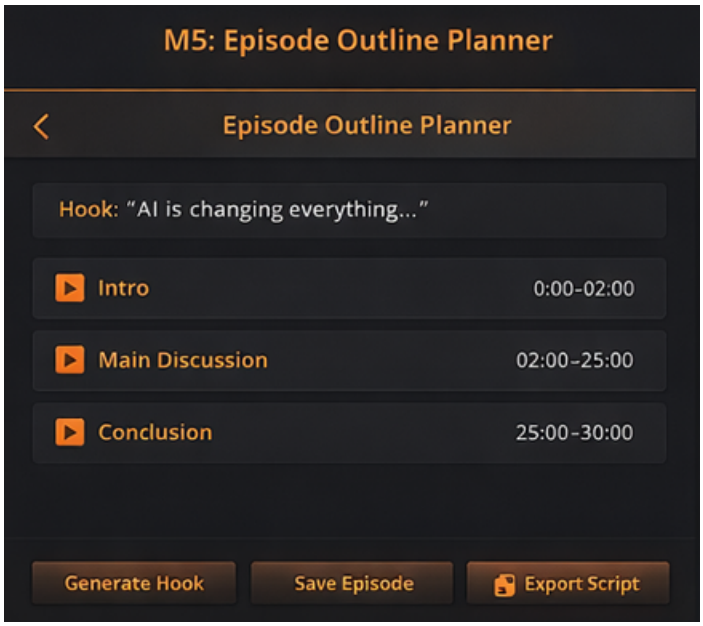
Web Mockups


M1: Login / Signup Screen
PodCraft
Email
Password
Login
Forgot Password?
Don't have an account?
Sign Up


M2: Dashboard
PodCraft
Welcome, John!
Enter Podcast Niche...
Guest Name
Generate Ideas
Recent Projects
Tech Podcast with AI Expert
Business Growth Episode

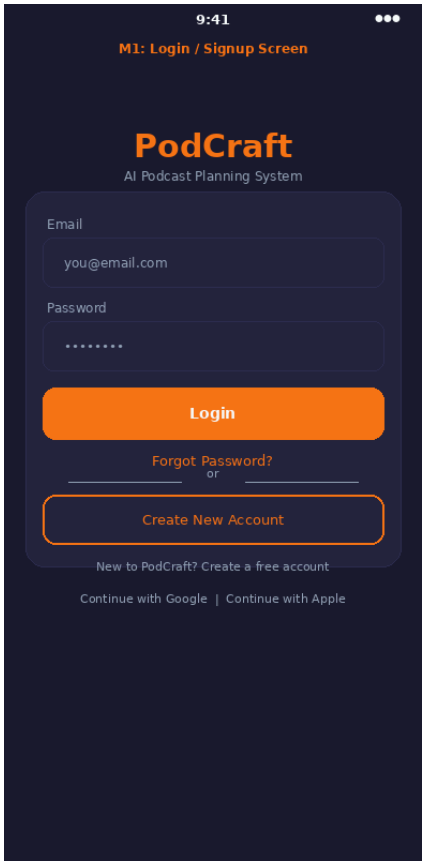

M3: Topic Generation
Topic Research Agent
Niche: Technology
Guest: AI Engineer
Generate Topics
Suggested Topics:
Future of AI
AI in Daily Life
Ethics in AI
Save
Export


M4: Question & Timestamp Generator
Question & Timestamp Generator
Topic: Future of AI
Q1: What inspired your journey? 0:00–05:00
Q2: What are the current trends? 05:00–15:00
Q3: Future predictions? 15:00–25:00
Edit
Save
Export

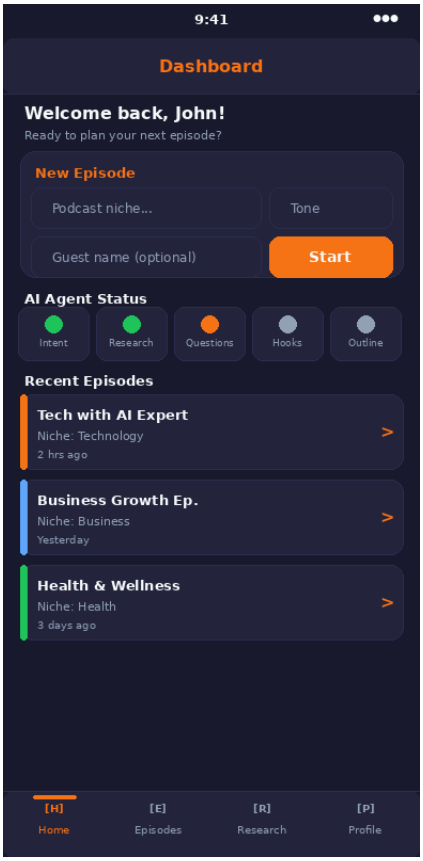


Mobile Mockups:

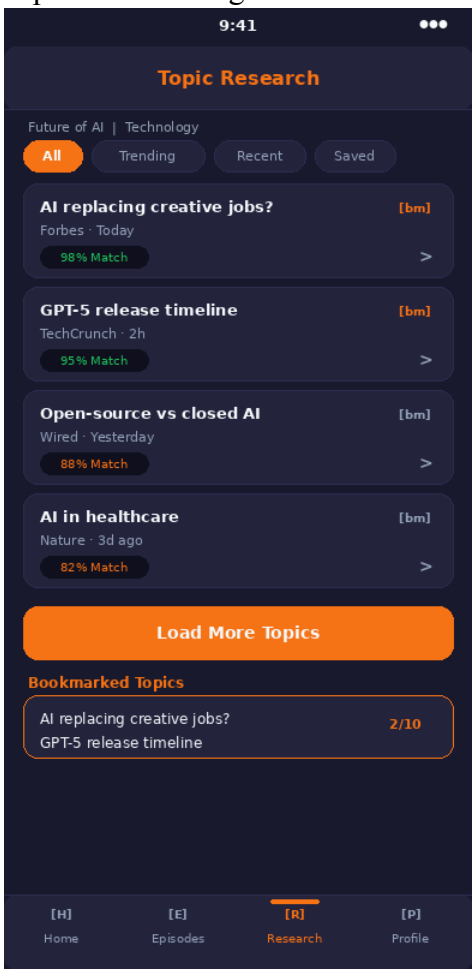
User Authentication:



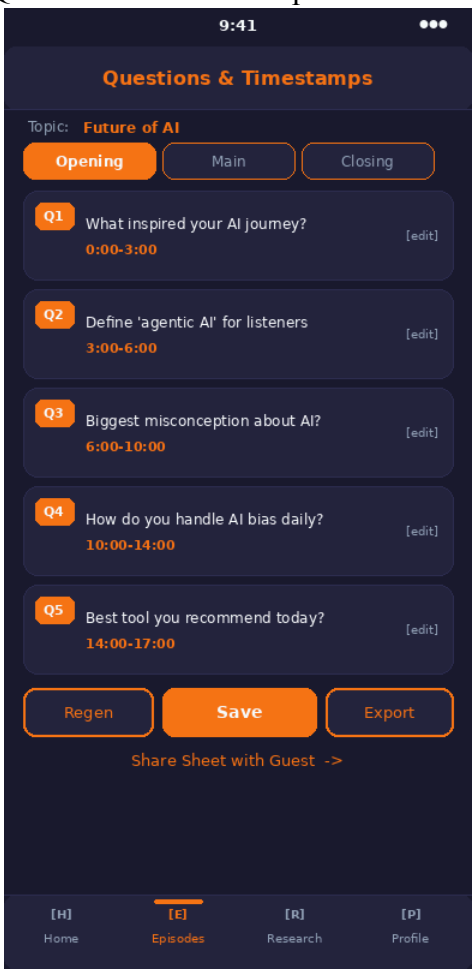
User Dashboard



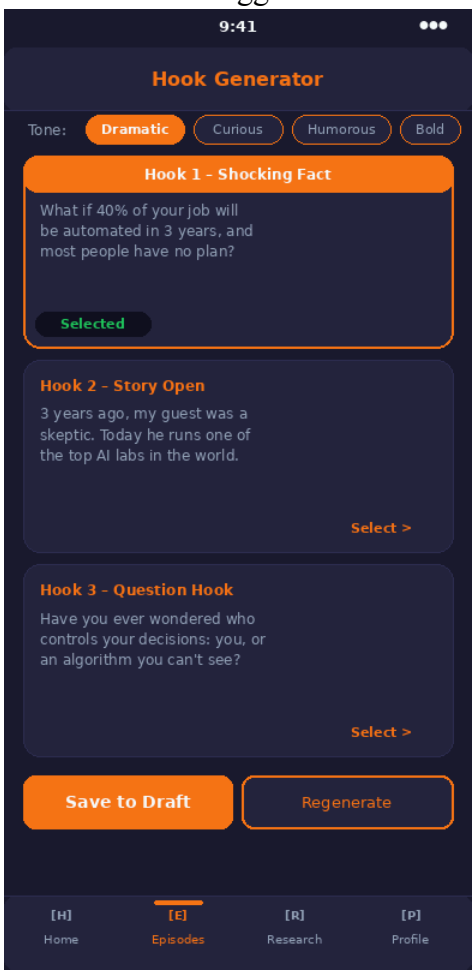
Topic Research Agent



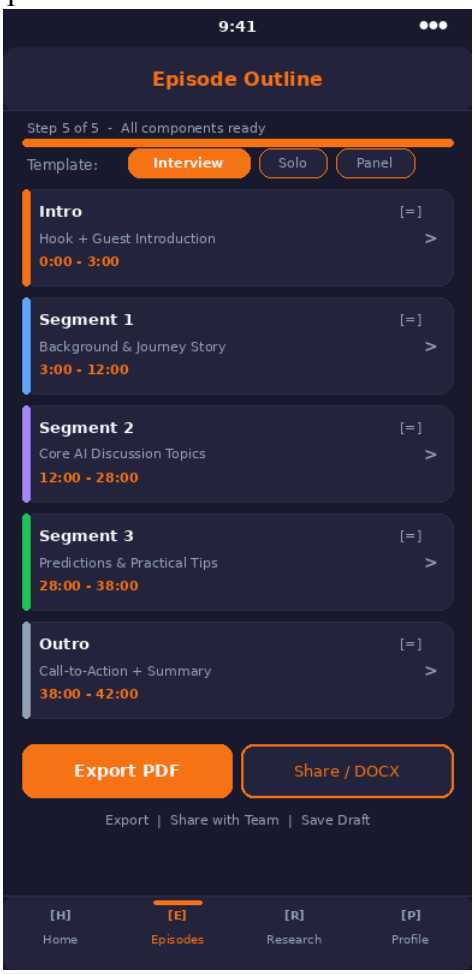
Question and Timestamp Generator



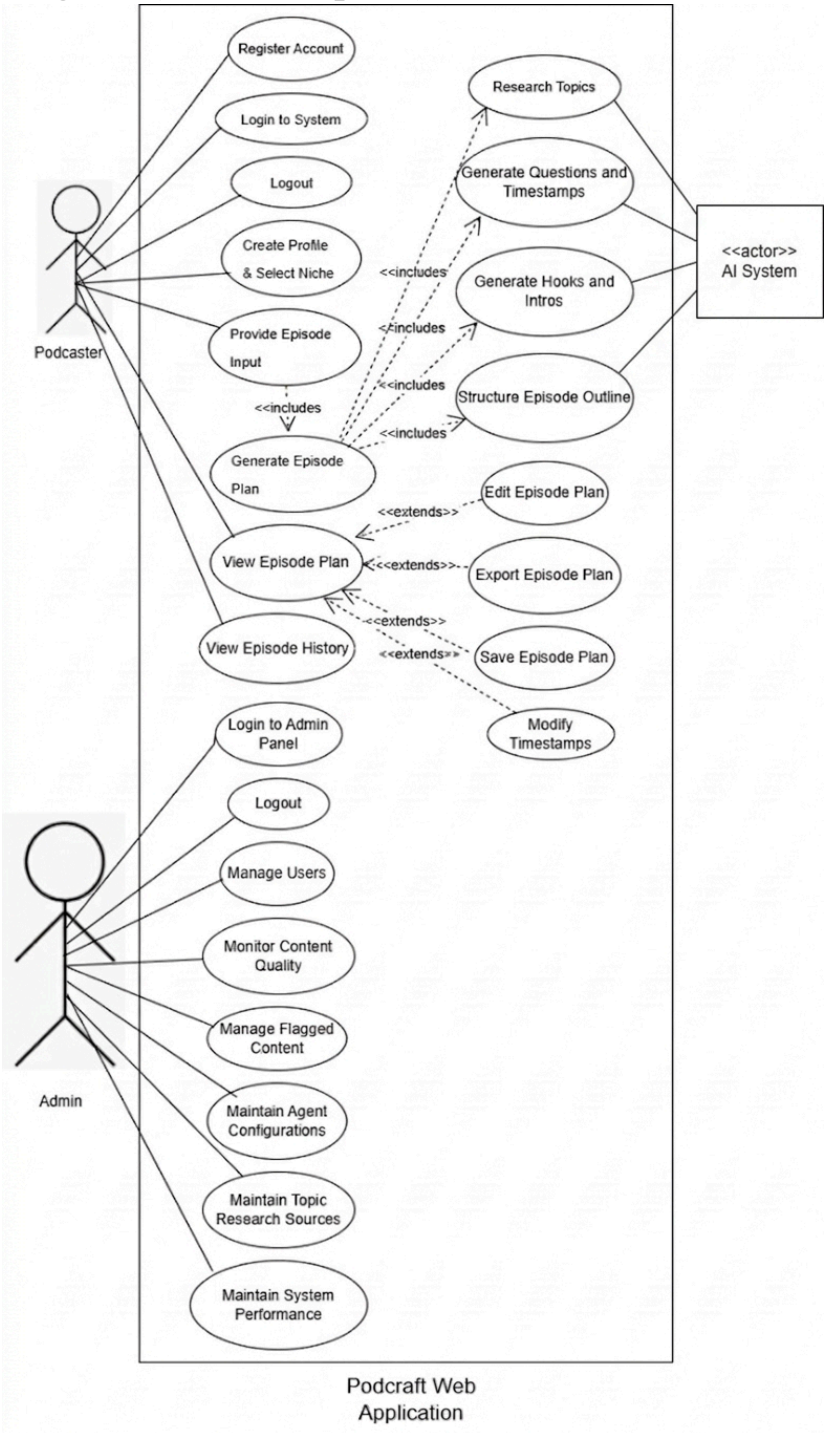
Hook and Intro Suggestion



Episode Outline Planner



2.5 Use Case Diagram (also Description)



The Use Case Diagram for PodCraft illustrates the interactions between four primary actors - the Podcaster, the Production Team Member, the System Administrator, and the AI Agent System - and the system's core use cases. The diagram is organized into five module-based boundaries corresponding to PodCraft's core modules.

The Podcaster actor interacts with the majority of use cases including providing niche/guest input, viewing research results, customizing questions, approving episode outlines, and exporting content. The Production Team Member shares most use cases with the Podcaster, with the addition of collaborative review and multi-episode management capabilities. The System Administrator manages user accounts, monitors agent logs, handles subscription plans, and configures API integrations.

The AI Agent System is modeled as a system-level actor responsible for autonomously executing research tasks, generating questions, producing hook suggestions, and building episode structures. Include relationships connect the Episode Outline Planner use case to both Topic Research Agent and Question Generator use cases, as the outline cannot be produced without their outputs. Extend relationships are used for optional behaviors such as "Export as PDF" extending "View Episode Outline."

2.5.1 Use Case Lists (Module-wise)

Module 1: User Authentication & Profile Management (Web)

UC ID	Use Case Name
M1-UC1	User Registration
M1-UC2	User Login
M1-UC3	Password Recovery
M1-UC4	Create Podcast Profile
M1-UC5	Select Podcast Niche
M1-UC6	View Episode History
M1-UC7	Add Guest Profile
M1-UC8	Edit Guest Profile
M1-UC9	Delete Saved Plan
M1-UC10	Update Account Settings
M1-UC11	Logout
M1-UC12	Delete Account

Module 2: User Intent & Guest Analyzer

UC ID	Use Case Name
M1-UC1	Provide Podcast Niche and Topic
M1-UC2	Submit Guest Profile
M1-UC3	Specify Episode Goals and Format
M1-UC4	Review Analyzed Intent Summary
M1-UC5	Edit and Refine Intent Inputs
M1-UC6	Save Podcast Profile for Reuse
M1-UC7	Switch Between Existing Podcast Profiles

M1-UC8	Delete Podcast Profile

Module 3: Topic Research Agent

UC ID	Use Case Name
M2-UC1	Initiate Topic Research
M2-UC2	View Researched Topics List
M2-UC3	Filter Topics by Relevance or Recency
M2-UC4	Bookmark Preferred Topics
M2-UC5	Reject Irrelevant Topics
M2-UC6	Request Additional Research Depth
M2-UC7	View Source References for Topics
M2-UC8	Export Research Summary

Module 4: Question & Timestamp Generator

UC ID	Use Case Name
M3-UC1	Generate Episode Questions
M3-UC2	Customize Question Count and Type
M3-UC3	Edit Individual Questions
M3-UC4	Reorder Questions via Drag-and-Drop
M3-UC5	Assign Time Allocations to Segments
M3-UC6	Auto-Generate Timestamp Structure
M3-UC7	Preview Full Q&A Timeline
M3-UC8	Export Questions and Timestamps as PDF/DOCX
M3-UC9	Share Question Sheet with Guest

Module 5: Hook & Intro Suggestion Agent

UC ID	Use Case Name
M4-UC1	Generate Multiple Hook Variants
M4-UC2	Select Preferred Hook Style
M4-UC3	Customize Hook Tone (e.g., dramatic, curious, humorous)
M4-UC4	RegenerateHooks with New Parameters
M4-UC5	Preview Hook with Episode Context
M4-UC6	Save Hook to Episode Draft
M4-UC7	View Hook Performance Suggestions
M4-UC8	Export Hook as Standalone Script

Module 6: Episode Outline Planner

UC ID	Use Case Name

M5-UC1	Generate Full Episode Outline
M5-UC2	View Outline in Structured Format
M5-UC3	Edit Outline Sections
M5-UC4	Reorder Outline Segments
M5-UC5	Add Custom Sections to Outline
M5-UC6	Apply Outline Templates
M5-UC7	Export Outline as PDF / DOCX / Notion
M5-UC8	Share Outline with Team Members
M5-UC9	Save Outline as Episode Draft

2.5.2 Use Cases (Tabular Format)

The following section provides detailed tabular use cases for all six modules of PodCraft. Each use case is documented with actors, trigger, preconditions, postconditions, normal flow, alternative flows, exceptions, and business rules.

Module 1: User Authentication & Profile Management

UC: M1-UC1 - User Registration

Use Case ID:	M1-UC1
Use Case Name:	User Registration
Actors:	Primary Actor: New User | Secondary Actor: Authentication System
Description:	A new user creates an account on PodCraft by providing their name, email address, and password. The system validates the input, creates the user record, and sends a verification email to activate the account.
Trigger:	User clicks 'Sign Up' on the PodCraft landing page or app.
Level:	High
Preconditions:	PRE-1: User does not already have a registered account with the provided email. PRE-2: PodCraft registration page is accessible.
Postconditions:	POST-1: A new user account is created in the database with a default free-tier subscription. POST-2: A verification email is sent to the provided email address. POST-3: User is redirected to an email verification notice screen.
Normal Flow:	1. User navigates to the Registration screen. 2. System displays the registration form (full name, email, password, confirm password). 3. User fills in all required fields. 4. User clicks 'Register'. 5. System validates inputs (email format, password strength, uniqueness). 6. System creates the user record and assigns a free-tier subscription.

	7. System sends a verification email. 8. System displays a confirmation message prompting the user to verify their email.
Alternative Flows:	1.1 Register via Google OAuth 1. User clicks 'Continue with Google'. 2. System redirects to Google OAuth consent screen. 3. User grants permission. 4. System creates account using Google profile data and skips email verification.
Exceptions:	1.0.E1: Email already registered — System displays error 'An account with this email already exists.' 1.0.E2: Password does not meet strength requirements — System displays inline validation message.
Business Rules:	BR-A1: Passwords must be at least 8 characters and include one uppercase letter, one number, and one special character. BR-A2: Each email address may be associated with only one PodCraft account.

UC: M1-UC2 - User Login

Use Case ID:	M1-UC2
Use Case Name:	User Login
Actors:	Primary Actor: Registered User | Secondary Actor: Authentication System
Description:	A registered user authenticates into PodCraft using their email and password credentials or via a third-party OAuth provider. Upon successful authentication, the user is granted access to their account dashboard.
Trigger:	User navigates to the PodCraft login page and clicks 'Log In'.
Level:	High
Preconditions:	PRE-1: User has a registered and verified PodCraft account. PRE-2: PodCraft authentication service is operational.
Postconditions:	POST-1: A valid session token (JWT) is issued and stored securely. POST-2: User is redirected to the Episode Dashboard. POST-3: Last login timestamp is updated in the database.
Normal Flow:	1. User navigates to the Login screen. 2. System displays the login form (email and password fields). 3. User enters credentials and clicks 'Log In'. 4. System validates credentials against the database. 5. System generates and issues a JWT session token. 6. User is redirected to the Episode Dashboard.
Alternative Flows:	1.1 Login via Google OAuth 1. User clicks 'Continue with Google'. 2. System redirects to Google OAuth and retrieves identity. 3. System matches identity to existing account and issues session token.

	1.2 Forgot Password 1. User clicks 'Forgot Password'. 2. System redirects to password recovery flow (see M1-UC3).
Exceptions:	1.0.E1: Invalid credentials — System displays error 'Incorrect email or password.' after each failed attempt. 1.0.E2: Account not verified — System prompts user to check their email for the verification link. 1.0.E3: Account locked after 5 failed attempts — System displays lockout message and prompts password reset.
Business Rules:	BR-A3: Session tokens expire after 24 hours of inactivity. BR-A4: After 5 consecutive failed login attempts, the account is temporarily locked for 15 minutes.

Module 2: User Intent & Guest Analyzer

UC: M2-UC1 - Provide Podcast Niche and Topic

Use Case ID:	M2-UC1
Use Case Name:	Provide Podcast Niche and Topic
Actors:	Primary Actor: Podcaster | Secondary Actor: AI Agent System
Description:	The Podcaster provides the podcast niche category (e.g., technology, personal development, true crime) and a specific episode topic or guest name. The system analyzes this input to configure downstream AI agents for targeted research and planning.
Trigger:	Podcaster clicks 'Start New Episode' and submits the niche and topic form.
Level:	High
Preconditions:	PRE-1: Podcaster is authenticated and logged into PodCraft. PRE-2: At least one active episode slot is available under the user's subscription plan.
Postconditions:	POST-1: Episode intent record is created in the database with niche and topic fields populated. POST-2: AI Agent System is triggered to begin topic research and guest profile analysis. POST-3: User is redirected to the Research Dashboard.
Normal Flow:	1.0 Provide Intent Input 1. Podcaster navigates to the New Episode screen. 2. System displays the Intent Input form with niche selector and topic/guest text fields. 3. Podcaster selects a niche from the dropdown (e.g., 'Business & Entrepreneurship'). 4. Podcaster enters episode topic or guest name in the text field. 5. Podcaster optionally provides episode duration and desired tone. 6. Podcaster clicks 'Analyze Intent'.

	7. System validates inputs and saves the episode intent record. 8. System dispatches AI agents with the collected parameters. 9. System displays a progress indicator confirming agent activation.
Alternative Flows:	1.1 Use Existing Podcast Profile 1. Podcaster selects a previously saved podcast profile. 2. System pre-fills niche and default tone from saved profile. 3. Return to step 4 of normal flow. 1.2 Topic from Trending Suggestions 1. Podcaster clicks 'Show Trending Topics' for the selected niche. 2. System displays AI-curated trending topics. 3. Podcaster selects a topic from the list. 4. Return to step 5. of normal flow.
Exceptions:	1.0.E1: Niche field is empty — System displays validation error 'Please select a niche.'. 1.0.E2: User has reached monthly episode limit — System displays upgrade prompt and terminates the use case.
Business Rules:	BR-1: Free-tier users are limited to 3 episode plans per calendar month. BR-2: The niche selection must come from a predefined taxonomy of categories maintained by the System Administrator.

UC: M2-UC2 - Submit Guest Profile

Use Case ID:	M2-UC2
Use Case Name:	Submit Guest Profile
Actors:	Primary Actor: Podcaster | Secondary Actor: AI Agent System
Description:	The Podcaster provides information about the episode guest, including name, profession, notable achievements, and social media or website links. The AI Guest Analyzer processes this to generate a guest profile summary that informs question generation and episode structuring.
Trigger:	Podcaster selects 'Add Guest' from the Episode Setup screen.
Level:	High
Preconditions:	PRE-1: An active episode plan has been created (M2-UC1 completed). PRE-2: Podcaster is authenticated.
Postconditions:	POST-1: Guest profile is stored and linked to the current episode. POST-2: AI analyzer generates a guest background summary visible on the Episode Dashboard POST-3: Question Generator agent is notified with guest context data.
Normal Flow:	1. Podcaster clicks 'Add Guest'. 2. System presents guest profile form. 3. Podcaster enters guest name, profession, and LinkedIn/website URL. 4. Podcaster optionally uploads a bio or pastes a brief description.

	5. Podcaster clicks 'Analyze Guest'. 6. AI Agent crawls public information from provided URL. 7. System displays generated guest profile summary for review. 8. Podcaster confirms or edits the profile summary. 9. System saves the confirmed guest profile.
Alternative Flows:	1.1 Manual Profile Entry 1. Podcaster skips URL and manually types the guest bio. 2. System uses the manual text directly without web crawling. 3. Return to step 7 of normal flow.
Exceptions:	1.0.E1: Provided URL is inaccessible — System notifies user and falls back to manual entry. 1.0.E2: Guest name field is empty — System displays validation error.
Business Rules:	BR-3: A maximum of 3 guests may be added per episode. BR-4: URL crawling is performed using the system's internal web scraper; guest data from crawl is not permanently stored beyond the episode lifecycle.

Module 3: Topic Research Agent

UC: M3-UC1 - Initiate Topic Research

Use Case ID:	M2-UC1
Use Case Name:	Initiate Topic Research
Actors:	Primary Actor: Podcaster | Secondary Actor: AI Topic Research Agent
Description:	The Podcaster triggers the autonomous Topic Research Agent, which scours the internet, academic sources, and trending news for relevant, up-to-date information matching the provided niche and topic. The agent returns a curated list of research findings.
Trigger:	Automatically triggered after M2-UC1 completes OR manually via 'Run Research' button on the Episode Dashboard.
Level:	High
Preconditions:	PRE-1: Episode intent (niche and topic) has been provided and saved. PRE-2: System has access to external search APIs (e.g., Perplexity, Bing Search).
Postconditions:	POST-1: Research results are stored in the episode's research repository. POST-2: Podcaster is notified via in-app notification that research is complete. POST-3: Research topics are presented in the Topic Research Dashboard.
Normal Flow:	1. System dispatches the Topic Research Agent with episode niche and topic as parameters. 2. Agent queries multiple sources: news APIs, academic databases, and social media trends. 3. Agent filters results by relevance score (minimum threshold: 0.65) and recency (last 6 months preferred). 4. Agent compiles top 10–15 topics with titles, summaries, and source URLs. 5. System stores results in database and marks research status as 'Complete'.

	6. System displays results on the Topic Research Dashboard.
Alternative Flows:	1.1 Manual Research Refresh 1. Podcaster clicks 'Refresh Research'. 2. System re-runs the agent with updated parameters. 3. New results replace or supplement existing research.
Exceptions:	1.0.E1: External API unavailable — System displays error and queues research for retry in 5 minutes. 1.0.E2: No results match relevance threshold — System displays 'No relevant topics found' and prompts manual topic entry.
Business Rules:	BR-5: Research results are cached for 24 hours per niche-topic combination to reduce API costs. BR-6: Premium users receive research from 5 source categories; free-tier users receive results from 2 source categories only.

UC: M3-UC4 - Bookmark Preferred Topics

Use Case ID:	M2-UC4
Use Case Name:	Bookmark Preferred Topics
Actors:	Primary Actor: Podcaster
Description:	The Podcaster reviews the list of AI-researched topics and marks specific topics as preferred by bookmarking them. Bookmarked topics are prioritized when the system generates questions and the episode outline.
Trigger:	Podcaster clicks the bookmark icon next to a topic in the Research Dashboard.
Level:	Medium
Preconditions:	PRE-1: Topic research has been completed and results are displayed. PRE-2: Podcaster is authenticated.
Postconditions:	POST-1: Selected topic's bookmark flag is set to true in the database. POST-2: Topic is visually distinguished (e.g., highlighted) in the dashboard. POST-3: Downstream agents (Question Generator, Outline Planner) prioritize bookmarked topics.
Normal Flow:	1. Podcaster views research results on the Topic Research Dashboard. 2. Podcaster clicks the bookmark icon next to a desired topic. 3. System toggles the bookmark state and visually highlights the topic. 4. System updates the priority weight of the topic in the episode context.
Alternative Flows:	1.1 Unbookmark Topic 1. Podcaster clicks bookmark icon on an already-bookmarked topic. 2. System removes highlight and resets priority weight to default.
Exceptions:	1.0.E1: Maximum bookmarks (10) reached — System notifies user to remove a bookmark before adding a new one.
Business Rules:	BR-7: A maximum of 10 topics may be bookmarked per episode. BR-8: At least one topic must be bookmarked before the Question Generator can be triggered.

Module 4: Question & Timestamp Generator

UC: M4-UC1 - Generate Episode Questions

Use Case ID:	M3-UC1
Use Case Name:	Generate Episode Questions
Actors:	Primary Actor: Podcaster | Secondary Actor: AI Question Generator Agent
Description:	The system autonomously generates a structured set of interview or discussion questions based on bookmarked topics, guest profile, and episode tone. Questions are categorized into opening, main discussion, and closing segments.
Trigger:	Podcaster clicks 'Generate Questions' on the Episode Planning screen, OR automatically triggered after bookmarking is completed.
Level:	High
Preconditions:	PRE-1: At least one topic has been bookmarked. PRE-2: Episode niche and optional guest profile data are available.
Postconditions:	POST-1: A structured question list is saved to the episode record. POST-2: Questions are displayed grouped by segment (Opening / Main / Closing). POST-3: Estimated time allocations are auto-assigned to each question.
Normal Flow:	1. Podcaster triggers question generation. 2. System sends bookmarked topics, guest profile, and tone preference to the AI agent. 3. AI generates 10–20 questions organized into three segments. 4. System assigns an estimated time (in minutes) to each question based on its complexity. 5. System displays generated questions in an editable list view. 6. Podcaster reviews the questions.
Alternative Flows:	1.1 Regenerate All Questions 1. Podcaster clicks 'Regenerate'. 2. System discards existing questions and re-runs the generation process. 1.2 Add Custom Question 1. Podcaster clicks 'Add Custom Question'. 2. System inserts a blank question field for manual entry.
Exceptions:	1.0.E1: AI API timeout — System displays error message and allows manual retry after 30 seconds.
Business Rules:	BR-9: Generated questions must cover at least 3 distinct topic areas from the bookmarked list. BR-10: Free-tier users may generate a maximum of 8 questions per episode; premium users have no limit.

UC: M4-UC6 - Auto-Generate Timestamp Structure

Use Case ID:	M3-UC6
Use Case Name:	Auto-Generate Timestamp Structure

Actors:	Primary Actor: Podcaster | Secondary Actor: AI Timestamp Agent
Description:	Based on the episode duration and the generated questions, the system automatically builds a timestamp structure showing when each segment should begin and end. This helps the podcaster plan pacing and maintain episode flow.
Trigger:	Podcaster clicks 'Generate Timestamps' after question list is finalized.
Level:	Medium
Preconditions:	PRE-1: Episode duration has been specified by the Podcaster. PRE-2: Questions have been generated and finalized.
Postconditions:	POST-1: Timestamp map is created and linked to the episode. POST-2: Timestamps are displayed alongside questions in the episode view. POST-3: Total estimated episode duration is recalculated and shown.
Normal Flow:	1. Podcaster provides total episode duration (e.g., 45 minutes). 2. System calculates time per segment proportionally based on question count and category. 3. System reserves 2–3 minutes for intro, 3–5 minutes for outro. 4. Timestamps are assigned to each question (HH:MM:SS format). 5. System displays the timestamp map on the Timeline View.
Alternative Flows:	1.1 Manual Timestamp Adjustment 1. Podcaster manually edits a timestamp field. 2. System recalculates downstream timestamps accordingly.
Exceptions:	1.0.E1: Episode duration not specified — System defaults to 60 minutes and notifies the user. 1.0.E2: Total question time exceeds episode duration — System warns user and suggests removing lower-priority questions.
Business Rules:	BR-11: Intro and outro segments are always allocated time and cannot be removed from the timestamp structure. BR-12: Minimum time per question is 1 minute; maximum is 15 minutes.

Module 5: Hook & Intro Suggestion Agent

UC: M5-UC1 - Generate Multiple Hook Variants

Use Case ID:	M5-UC1
Use Case Name:	Generate Multiple Hook Variants
Actors:	Primary Actor: Podcaster | Secondary Actor: AI Hook Suggestion Agent
Description:	The AI Hook Suggestion Agent generates 3–5 distinct episode opening hook variants designed to capture listener attention within the first 30 seconds. Each variant reflects a different rhetorical approach (e.g., shocking statistic, storytelling opener, provocative question).
Trigger:	Podcaster clicks 'Generate Hooks' on the Hook & Intro screen.
Level:	High
Preconditions:	PRE-1: Episode topic and niche are confirmed. PRE-2: At least one topic is bookmarked from the research phase.

Postconditions:	POST-1: 3–5 hook variants are stored and displayed. POST-2: Each hook is tagged with its rhetorical style and estimated read-aloud duration.
Normal Flow:	1. Podcaster navigates to the Hook & Intro screen. 2. Podcaster selects preferred tone (e.g., dramatic, curious, humorous). 3. System passes topic, niche, guest name, and tone to the Hook Agent. 4. Agent generates 3–5 hook variants with distinct styles. 5. System displays hooks with style labels and word counts. 6. Podcaster reads and selects a preferred hook.
Alternative Flows:	1.1 Regenerate Specific Hook 1. Podcaster clicks 'Regenerate' on a specific hook variant. 2. Agent creates a new variant for that style only.
Exceptions:	1.0.E1: AI Agent fails to generate hooks after 3 attempts — System offers a manual hook editor.
Business Rules:	BR-13: Each hook variant must be between 30 and 120 words. BR-14: Hooks must not include copyrighted content or reproduce verbatim text from research sources.

UC: M5-UC2 - Select Preferred Hook Style

Use Case ID:	M5-UC2
Use Case Name:	Select Preferred Hook Style
Actors:	Primary Actor: Podcaster
Description:	After reviewing the generated hook variants, the Podcaster selects their preferred rhetorical style for the episode opening. The selected hook is saved to the episode draft and used as the intro in the final outline.
Trigger:	Podcaster clicks 'Select' on one of the generated hook variants on the Hook & Intro screen.
Level:	Medium
Preconditions:	PRE-1: At least one hook variant has been successfully generated (M5-UC1 completed). PRE-2: Podcaster is authenticated and viewing the Hook & Intro screen.
Postconditions:	POST-1: The selected hook is flagged as the active hook and saved to the episode draft. POST-2: The Episode Outline Planner is notified with the selected hook content. POST-3: The selected hook is visually highlighted and all other variants are marked as inactive.
Normal Flow:	1. Podcaster reviews the list of generated hook variants on the Hook & Intro screen. 2. Podcaster clicks 'Select' on the preferred hook variant. 3. System saves the selected hook to the episode draft record.

	4. System visually highlights the selected hook and dims the remaining variants. 5. System notifies the Episode Outline Planner agent with the selected hook content. 6. Podcaster receives a confirmation message: 'Hook saved to episode draft.'
Alternative Flows:	1.1 Change Hook Selection 1. Podcaster clicks 'Select' on a different hook variant after a prior selection. 2. System replaces the previously selected hook with the newly selected one in the episode draft. 3. Episode Outline Planner is updated with the new hook content.
Exceptions:	1.0.E1: No hook variants available — System prevents selection and prompts the user to generate hooks first (see M5-UC1).
Business Rules:	BR-13: Each hook variant must be between 30 and 120 words. BR-17: Only one hook variant may be selected as active per episode at any given time.

Module 6: Episode Outline Planner

UC: M6-UC1 - Generate Full Episode Outline

Use Case ID:	M6-UC1
Use Case Name:	Generate Full Episode Outline
Actors:	Primary Actor: Podcaster | Secondary Actor: AI Outline Planner Agent
Description:	The AI Outline Planner Agent synthesizes all prior planning outputs — research topics, guest profile, questions, timestamps, and selected hook — into a cohesive, structured episode outline. The outline defines the full flow of the episode from intro to outro.
Trigger:	Podcaster clicks 'Build Outline' after completing hook selection and timestamp approval.
Level:	High
Preconditions:	PRE-1: Questions and timestamps are finalized. PRE-2: A hook has been selected or a custom intro has been written. PRE-3: Research topics have been reviewed and bookmarked.
Postconditions:	POST-1: Full episode outline is saved to the database. POST-2: Outline is accessible from the Episode Dashboard. POST-3: Outline is available for export in PDF, DOCX, and Notion formats.
Normal Flow:	1. Podcaster clicks 'Build Outline'. 2. System collects all finalized episode components. 3. AI Outline Agent organizes sections: Intro → Segment 1 → Segment 2 → [...] → Outro. 4. Each section is populated with questions, topic summaries, and timestamps. 5. System renders the outline in a structured, readable format. 6. Podcaster reviews the complete outline. 7. Podcaster approves or edits the outline.

Alternative Flows:	1.1 Apply Outline Template 1. Podcaster selects a predefined template (e.g., 'Interview', 'Solo Episode', 'Panel Discussion'). 2. System structures outline sections according to template. 3. Agent fills in content within the template structure.
Exceptions:	1.0.E1: No questions available — System prevents outline generation and directs user to complete question generation first. 1.0.E2: No hook selected — System warns user but permits outline generation with a blank intro placeholder.
Business Rules:	BR-15: The outline must contain a minimum of 3 sections (Intro, Main, Outro). BR-16: Outline generation consumes 1 unit from the user's monthly AI generation quota.

UC: M6-UC2 - View Outline in Structured Format

Use Case ID:	M6-UC2
Use Case Name:	View Outline in Structured Format
Actors:	Primary Actor: Podcaster
Description:	After the episode outline has been generated, the Podcaster can view the complete outline in a structured, readable format. The outline is organized into labeled sections (Intro, Main Segments, Outro) with associated questions, topic summaries, and timestamps displayed for each segment.
Trigger:	Podcaster navigates to the Episode Dashboard and clicks 'View Outline'.
Level:	Medium
Preconditions:	PRE-1: A full episode outline has been generated and saved (M6-UC1 completed). PRE-2: Podcaster is authenticated.
Postconditions:	POST-1: The full outline is rendered and visible to the Podcaster in the structured outline view. POST-2: Each section is displayed with its title, associated questions, topic summary, and timestamp. POST-3: Export and edit options are accessible from the outline view.
Normal Flow:	1. Podcaster clicks 'View Outline' from the Episode Dashboard. 2. System retrieves the saved episode outline from the database. 3. System renders the outline in a structured view: Intro → Main Segment(s) → Outro. 4. Each section displays: section title, topic summary, associated questions, and timestamps. 5. Podcaster scrolls through the full outline. 6. Podcaster can access Edit, Export, and Share options from the toolbar.
Alternative Flows:	1.1 Collapse / Expand Sections 1. Podcaster clicks on a section header to collapse or expand its content. 2. System toggles the visibility of that section's detail rows without navigating away.

Exceptions:	1.0.E1: Outline data is unavailable or corrupted — System displays an error and offers to regenerate the outline. 1.0.E2: No outline exists yet — System redirects user to the outline generation screen (M6-UC1).
Business Rules:	BR-15: The outline must contain a minimum of 3 sections (Intro, Main, Outro). BR-18: The structured view is read-only by default; Podcaster must click 'Edit Outline' to enter edit mode (see M6-UC3).

2.6 Functional Requirements

The following functional requirements are derived from the identified use cases and UI mockups. Each requirement is uniquely identified, clearly stated, and accompanied by applicable business rules.

Table 2.1: Functional Requirements - Module 1: User Authentication & Profile Management

Feature (from UI / Module)	Functional Requirement (FR-ID: Statement)	Business Rule
User Registration	FR1.1: The system shall allow new users to register by providing a full name, email address, and password, and shall send a verification email upon successful registration.	BR-A2: Each email may be associated with only one account.
OAuth Registration	FR1.2: The system shall support user registration and login via Google OAuth 2.0, bypassing email verification when using a verified Google account.	—
User Login	FR1.3: The system shall authenticate registered users using their email and password, issue a JWT session token upon success, and update the last login timestamp.	BR-A3: Session tokens expire after 24 hours of inactivity.
Login Failure Handling	FR1.4: The system shall display an error message after each failed login attempt and temporarily lock the account for 15 minutes after 5 consecutive failed attempts.	BR-A4: Account locked for 15 minutes after 5 failed attempts.
Password Recovery	FR1.5: The system shall allow users to initiate password recovery by entering their registered email, and shall send a time-limited password reset link valid for 30 minutes.	—
Podcast Profile Management	FR1.6: The system shall allow authenticated users to create, view, edit, and delete their podcast profile including display name and niche preferences.	BR-A1: Password must be at least 8 characters with uppercase, number, and special character.

Account Settings	FR1.7: The system shall allow users to update their account settings including email address, password, and notification preferences.	—
Logout	FR1.8: The system shall allow users to log out, immediately invalidating the active session token and redirecting to the login screen.	—

Table 2.2: Functional Requirements - Module 2: User Intent & Guest Analyzer

Feature (from UI / Module)	Functional Requirement (FR-ID: Statement)	Business Rule
Submit Podcast Niche	FR2.1: The system shall allow the Podcaster to select a podcast niche from a predefined taxonomy of at least 20 categories.	BR-2: Niche categories are managed by the System Administrator.
Episode Topic Input	FR2.2: The system shall provide a text field allowing the Podcaster to enter a specific episode topic or guest name (max 200 characters).	—
Intent Analysis Trigger	FR2.3: The system shall initiate the AI Intent Analyzer agent within 3 seconds of the Podcaster submitting the episode intent form.	—
Podcast Profile Management	FR2.4: The system shall allow the Podcaster to save, load, edit, and delete up to 10 named podcast profiles.	Free-tier users may save up to 2 profiles only.
Episode Tone Selection	FR2.5: The system shall provide tone selection options (Casual, Professional, Humorous, Serious) that influence AI-generated content throughout the episode.	—
Episode Limit Enforcement	FR2.6: The system shall prevent free-tier users from creating more than 3 episode plans per calendar month and display an upgrade prompt when the limit is reached.	BR-1: Monthly limit resets on the first day of each calendar month.
Guest Profile Submission	FR2.7: The system shall allow the Podcaster to submit guest information including name, profession, and URL, and shall generate an AI guest summary within 10 seconds.	BR-3: Maximum 3 guests per episode. BR-4: Crawled data not stored permanently.
Manual Guest Bio Entry	FR2.8: The system shall allow the Podcaster to manually enter a guest biography as an alternative to URL-based extraction.	—

Table 2.3: Functional Requirements - Module 3: Topic Research Agent

Feature (from UI / Module)	Functional Requirement (FR-ID: Statement)	Business Rule
Research Initiation	FR3.1: The system shall automatically trigger the Topic Research Agent upon successful submission of episode intent, delivering research results within 30 seconds.	BR-5: Results are cached for 24 hours per niche-topic pair.

Research Result Display	FR3.2: The system shall display researched topics in a paginated list showing title, summary (max 3 sentences), source name, and publication date.	—
Topic Filtering	FR3.3: The system shall allow the Podcaster to filter research results by relevance score and recency (last 30 days / 6 months / all time).	—
Topic Bookmarking	FR3.4: The system shall allow the Podcaster to bookmark up to 10 topics per episode. Bookmarked topics shall be visually highlighted and prioritized in downstream agents.	BR-7: Max 10 bookmarks. BR-8: At least 1 bookmark required before question generation.
Topic Rejection	FR3.5: The system shall allow the Podcaster to reject irrelevant topics; rejected topics shall not appear in subsequent research refreshes unless explicitly re-included.	—
Source Reference Viewing	FR3.6: The system shall provide a link to the original source URL for each research topic, opening in a new browser tab.	—
Tier-Based Research Depth	FR3.7: The system shall provide research results from 5 source categories for premium users and 2 source categories for free-tier users.	BR-6: Source categories: News, Academic, Social Media, Industry Blogs, Podcasts.
Research Export	FR3.8: The system shall allow the Podcaster to export the full research summary as a PDF document.	—

Table 2.4: Functional Requirements - Module 4: Question & Timestamp Generator

Feature (from UI / Module)	**Functional Requirement (FR-ID: Statement)**	**Business Rule**
Question Generation	FR4.1: The system shall generate 10–20 structured questions based on bookmarked topics, guest profile, and tone, organized into Opening (3), Main (10–14), and Closing (3) segments.	BR-9: Questions must cover at least 3 distinct topic areas.
Question Count Customization	FR4.2: The system shall allow the Podcaster to specify the desired number of questions (minimum 5, maximum 30 for premium users; maximum 8 for free-tier users).	BR-10: Free-tier max is 8 questions.
Question Editing	FR4.3: The system shall allow the Podcaster to directly edit the text of any generated question inline without regenerating the full question set.	—
Question Reordering	FR4.4: The system shall support drag-and-drop reordering of questions within and across segments.	—

Timestamp Auto-Generation	FR4.5: The system shall automatically generate a timestamp structure (HH:MM:SS) for each segment based on the total episode duration and question count, reserving 2–3 minutes for intro and 3–5 minutes for outro.	BR-11: Intro/outro are non-removable. BR-12: Min 1 min, Max 15 min per question.
Manual Timestamp Editing	FR4.6: The system shall allow the Podcaster to manually override any auto-generated timestamp; the system shall then recalculate all subsequent timestamps accordingly.	—
Question-Guest Sharing	FR4.7: The system shall generate a shareable link to the question sheet that can be sent to the episode guest without requiring the guest to create an account.	—
Questions Export	FR4.8: The system shall allow the Podcaster to export questions and timestamps in PDF and DOCX formats.	—

Table 2.5: Functional Requirements - Module 5: Hook & Intro Suggestion Agent

Feature (from UI / Module)	**Functional Requirement (FR-ID: Statement)**	**Business Rule**
Hook Generation	FR5.1: The system shall generate 3–5 distinct hook variants upon user request, each reflecting a different rhetorical style and tagged accordingly (e.g., Shocking Fact, Story Open, Question Hook).	BR-14: Hooks must not include copyrighted or verbatim source text.
Hook Tone Customization	FR5.2: The system shall allow the Podcaster to select a tone (Dramatic, Curious, Humorous, Inspirational) prior to hook generation, which shall influence the style of all generated variants.	—
Hook Length Compliance	FR5.3: The system shall ensure each generated hook variant is between 30 and 120 words. The system shall display word count alongside each variant.	BR-13: Length range 30–120 words.
Individual Hook Regeneration	FR5.4: The system shall allow the Podcaster to regenerate a single hook variant without affecting the other generated variants.	—
Hook Selection & Saving	FR5.5: The system shall allow the Podcaster to select exactly one hook variant to save to the episode draft. The selected hook shall be used as the episode intro in the outline.	—
Hook Preview	FR5.6: The system shall provide a read-aloud preview feature showing how the selected hook would sound in context with the episode title and first question.	—

Hook Export	FR5.7: The system shall allow the Podcaster to export the selected hook as a standalone plain-text script file.	—

Table 2.6: Functional Requirements - Module 6: Episode Outline Planner

Feature (from UI / Module)	Functional Requirement (FR-ID: Statement)	Business Rule
Outline Generation	FR6.1: The system shall generate a full episode outline synthesizing research topics, questions, timestamps, and selected hook, organized into a minimum of 3 sections (Intro, Main Segments, Outro).	BR-15: Minimum 3 sections required.
Outline Section Editing	FR6.2: The system shall allow the Podcaster to edit the title, description, and content of any section within the generated outline inline.	—
Outline Reordering	FR6.3: The system shall support drag-and-drop reordering of outline sections.	—
Custom Section Addition	FR6.4: The system shall allow the Podcaster to add custom sections to the outline at any position, specifying a section title and optional notes.	—
Template Application	FR6.5: The system shall provide at least 3 outline templates (Interview, Solo Episode, Panel Discussion) that the Podcaster can apply to restructure the outline.	—
Multi-Format Export	FR6.6: The system shall allow the Podcaster to export the episode outline in PDF, DOCX, and Notion-compatible Markdown formats.	—
Team Sharing	FR6.7: The system shall allow Production Team users to share the episode outline with team members via an in-app notification and shareable link.	Only available to Production Team user class.
Episode Draft Saving	FR6.8: The system shall automatically save outline edits every 30 seconds to prevent data loss. Manual save is also available.	BR-16: Generation consumes 1 AI quota unit.

2.7 Non-Functional Requirements

The following non-functional requirements specify the quality attributes of PodCraft. All requirements are expressed in quantitative, verifiable terms.

2.7.1 Reliability

- REL-1: The system shall achieve a minimum uptime of 99.5% measured over any 30-day rolling period, excluding scheduled maintenance windows of no more than 2 hours per month.
- REL-2: The system shall implement automatic retry logic for failed AI agent API calls, retrying up to 3 times with exponential backoff (2s, 4s, 8s intervals) before displaying an error to the user.
- REL-3: All user-generated episode data shall be automatically backed up to a secondary cloud region every 6 hours. Recovery Point Objective (RPO) shall not exceed 6 hours, and Recovery Time Objective (RTO) shall not exceed 1 hour.
- REL-4: The system shall maintain data integrity across all AI agent workflows; if any agent in a multi-step pipeline fails, previously completed steps shall be preserved and the failed step shall be re-triggerable independently.

2.7.2 Usability

- USE-1: A first-time user shall be able to complete a full episode plan (from niche input to outline export) within 15 minutes of initial registration, without requiring any external documentation.
- USE-2: The system shall display contextual inline help tooltips for all AI-generated outputs, explaining in plain language what the content represents and how to edit it.
- USE-3: The system shall support keyboard-only navigation for all primary actions (episode creation, question editing, outline export) to meet WCAG 2.1 Level AA accessibility standards.
- USE-4: All AI generation actions shall provide real-time progress indicators (spinner or progress bar) and display completion confirmation messages within 2 seconds of result availability.

2.7.3 Performance

- PER-1: 95% of all page loads and navigation transitions within the web application shall complete within 2 seconds over a 10 Mbps or faster internet connection.
- PER-2: Topic research results shall be delivered to the user within 30 seconds for 90% of requests under normal load conditions (fewer than 500 concurrent users).
- PER-3: Question generation shall complete within 15 seconds for 95% of requests. Episode outline generation shall complete within 20 seconds for 95% of requests.
- PER-4: The system shall support a minimum of 500 simultaneous active users without degradation exceeding 10% in response times compared to single-user benchmarks.
- PER-5: Mobile application cold start time shall not exceed 4 seconds on devices with 3GB RAM or more running Android 10+ or iOS 14+.

2.7.4 Security

- SEC-1: All data transmissions between the client and server shall be encrypted using TLS 1.3 or higher. HTTP traffic shall be automatically redirected to HTTPS.
- SEC-2: User passwords shall be hashed using bcrypt with a minimum cost factor of 12 before storage. Plain-text passwords shall never be stored or logged.
- SEC-3: User accounts shall be locked for 15 minutes after 5 consecutive failed login attempts. Account lockout events shall be logged and the account owner notified via email.
- SEC-4: All AI-generated content shall be screened through a content moderation layer before display to detect and block harmful, offensive, or inappropriate content.
- SEC-5: Third-party API keys (OpenAI, Perplexity, Stripe) shall be stored in environment variables using a secrets management service (e.g., AWS Secrets Manager) and shall never be exposed in client-side code.

2.8 External Interface Requirements

2.8.1 User Interface Requirements

- UI-1: The web application shall follow a clean, minimal design language with a primary color palette of dark navy (#0F172A), accent blue (#3B82F6), and neutral gray (#F8FAFC) for consistent visual identity.
- UI-2: The interface shall use the Inter typeface at base size 14px for body text, 16–24px for headings, with sufficient contrast ratios meeting WCAG 2.1 AA standards (minimum 4.5:1 for normal text).
- UI-3: All primary actions (Generate, Save, Export) shall be accessible within 2 clicks from any screen. Navigation shall include a persistent left sidebar with module icons and labels.
- UI-4: The system shall display real-time agent status indicators (Idle, Running, Complete, Error) for all 5 AI agents, visible from the Episode Dashboard.
- UI-5: The mobile interface shall implement a bottom navigation bar with icons for Home, Episodes, Research, and Profile, replacing the sidebar used on web.
- UI-6: All form inputs shall include placeholder text, character counters where applicable, and inline validation error messages displayed in red (#EF4444) below the relevant field.

2.8.2 Software Interfaces

Interface ID	System/Service	Purpose
SI-1	OpenAI GPT-4 API (v1)	Natural language generation for questions, hooks, outlines, and summaries.
SI-2	Perplexity / Bing Search API	Real-time internet research and trending topic retrieval for the Topic Research Agent.

SI-3	PostgreSQL 15	Primary relational database for storing user accounts, episodes, research results, and AI outputs.
SI-4	Redis 7.x	Caching layer for AI research results (24-hour TTL) and real-time agent task queue management.
SI-5	Stripe API (v2023-10-16)	Subscription billing, payment processing, and invoice management for premium plans.
SI-6	Firebase Auth / Google OAuth 2.0	Third-party social sign-in (Google, Apple ID) and JWT-based session management.
SI-7	SendGrid Email API	Transactional email delivery for account verification, password reset, and episode share notifications.

2.8.3 Hardware Interfaces

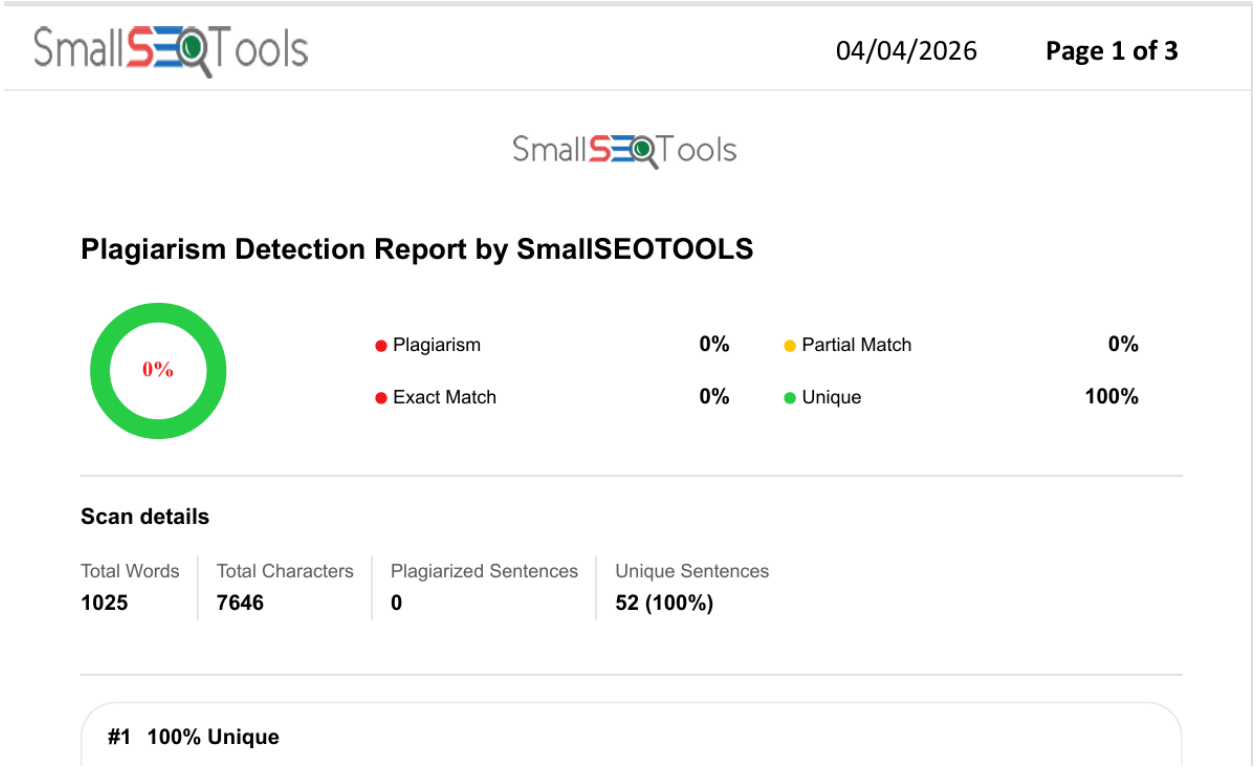
- HI-1: PodCraft is a software-only application and does not require dedicated hardware components. It operates on standard commodity cloud server hardware (x86-64 architecture) provisioned through cloud providers.
- HI-2: On mobile devices, the system shall interface with the device microphone through the OS media API to support future features such as voice-to-topic input. Microphone access shall require explicit user permission.
- HI-3: The system shall be compatible with standard input devices (keyboard, mouse, touchscreen) and shall not require any proprietary hardware accessories.

2.8.4 Communications Interfaces

- CI-1: All client-server communication shall occur over HTTPS using RESTful API endpoints. Responses shall be formatted as JSON with consistent status codes and error payloads.
- CI-2: Real-time agent progress updates shall be communicated to the client via WebSocket connections (Socket.IO). The system shall fall back to HTTP long-polling if WebSocket is unavailable.
- CI-3: The system shall send transactional email notifications (episode complete, subscription renewal, password reset) via the SendGrid SMTP relay using a verified PodCraft domain.
- CI-4: The system shall support push notifications on mobile devices for AI task completion events, using Firebase Cloud Messaging (FCM) for Android and Apple Push Notification Service (APNs) for iOS.

Plagiarism Report:

The turnitin report required one working day to be generated, so we have mailed our document for it, meanwhile we have added another plagiarism report created using an online tool.



References

- Wiegers, K. E., & Beatty, J. (2013). Software Requirements (3rd ed.). Microsoft Press.
- Sommerville, I. (2020). Software Engineering (10th ed.). Pearson Education Limited.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- OpenAI. (2024). GPT-4 API Reference. https://platform.openai.com/docs/api-reference
- Perplexity AI. (2025). Perplexity API Documentation. https://docs.perplexity.ai/
- Stripe. (2025). Stripe API Reference Documentation. https://stripe.com/docs/api
- Pohl, K., & Rupp, C. (2022). Requirements Engineering Fundamentals: A Study Guide for the Certified Professional for Requirements Engineering Exam. Rocky Nook.
- Yayici, E. (2022). UX Design and Usability Mentor Book. UXness Publications.
- Google Firebase. (2025). Firebase Authentication Documentation. https://firebase.google.com/docs/auth
- Redis Labs. (2025). Redis Documentation. https://redis.io/documentation

Work Division:

Asna Hammad (FA23-BDS-051)

- **Word Doc:** Whole chapter 1(1.1-1.11), 2.4 mockups for mobile app, Use case diagram, 2.5.1: Use case lists + 2.5.2: tables + 2.6: functional requirements for first 2 modules, plagiarism report for word.

  Word doc formatting
- **Presentation:** Slides 1-5, 15, 16, 19-29, 31.

Awais Ashraf (FA23-BDS-053)

- **Word doc:** Chapter 2(2.1- 2.4), mockups for web, 2.5: use case description, 2.5.1/2 (use case lists and tables for remaining 4 modules), 2.6 (Functional Requirements tables for remaining 4 modules), 2.7, 2.8, references, plagiarism report for pptx.
  pptx formatting
- **Presentation:** slides 9-14, 17, 18, 30, 32, 33